Lab 8: FDs and Normalization

In this lab you will learn about functional dependencies—constraints that hold over the data—and learn how to use them to improve schema design.

Objectives:

- Determine if functional dependencies hold on a relation.
- Identify functional dependencies in a table with data.
- Compute closures on determinants of functional dependencies.
- Identify super keys, candidate keys, and primary keys on a relation given functional dependencies.
- Determine if a relation is in BCNF given functional dependencies.
- Decompose a relation using the BCNF algorithm.
- Determine if a decomposition is lossy or lossless.

Few of the exercises in this lab activity are autograded. For most of them, we will work through the problems in class.

Part 1: Checking if FDs hold

Functional dependencies define constraints over data. A functional dependency may or may not hold over a data instance. A functional dependency A→B holds over a data instance of a relation R, if for any two tuples in R that share the same value for attribute A, the tuples also share the same value for attribute B.

Consider the following instance of a relation R(A,B,C,D):

A	B	C	D
1	2	5	2
2	2	4	2
2	3	5	7

Consider each of the following functional dependencies and determine whether they hold on this data instance.

1. A → B
2. B → A
3. B → C
4. B → D
5. D → B
6. D → A
7. AB → C
8. AB → D
9. BD → C

You should first think about these dependencies and decide whether they hold based on the definition.

Q1: Now we will think about determining whether these dependencies hold programmatically. Create a new database in postgres and add this small table. You can do so with these commands:

```
CREATE TABLE R (A INT, B INT, C INT, D INT);
INSERT INTO R VALUES (1,2,5,2), (2,2,4,2), (2,3,5,7);
```

Q1a: Write a SQL query over the table R that returns all values of A that appear with more than one distinct value of C. If the result is empty, A → C holds in this instance; otherwise each returned value witnesses a violation. Submit your SQL query in file Q1a.sql.

Q1b: Write a SQL query over the table R that returns all values of B that appear with more than one distinct value of D. If the result is empty, B → D holds in this instance; otherwise each returned value witnesses a violation. Submit your SQL query in file Q2a.sql.

Part 2: Computing closures

Consider the following schema and set of functional dependencies:

R(A,B,C,D,E,F,G) with functional dependencies:

A⟶D
D⟶C
F ⟶ EG
DC ⟶ BF

- Compute the appropriate closures to determine whether each of the following dependencies hold:

  A⟶E
  AC⟶E
  C⟶F
  D⟶G
  FC⟶B

- Can you identify the key of relation R?

Step 3: BCNF decomposition (normalization)

1. Is relation R from step 2 in BCNF? Why? If it is not in BCNF, decompose it.

2. Suppose we are told that R(A, B, C, D) is in BCNF, and that exactly 3 out of the 4 FDs listed below hold for R. Choose the FD that R does not satisfy, and explain your reasoning.

1: A⟶BCD
2: BC ⟶ A
3: CD⟶B
4: D⟶C

3. BCNF decomposition does not always yield the same result. The outcome depends on the order you choose to decompose on. Can you come up with an example (a relation R and a set of FDs) that demonstrates this?

Part 4: Lossless vs lossy decompositions

Consider the following decomposition of R from Part 1 into two relations:

R1(A, B) and R2(B, C, D)

You can create this decomposition in your database as follows:

```
CREATE TABLE R1 AS (SELECT DISTINCT A, B FROM R);
CREATE TABLE R2 AS (SELECT DISTINCT B, C, D FROM R);
```

What do you observe when you join tables R1 and R2 on their common attributes (B in this case)?

Q2: Write a SQL query that: computes the join of R1 and R2, and returns only tuples that appear in the join but *not* in the original R. These are the **spurious tuples**. If the result is empty, the decomposition is lossless. Submit your SQL query in file Q2.sql.

Hint: build the join in a subquery, then LEFT JOIN back to R and filter for rows where R.A IS NULL.

Now reason about the following decocompositions:

1. Consider relation R from Part 2. Is the decomposition of R into R1(A,D) and R2(B,C,D,E,F,G) in BCNF? Is it a lossless decomposition?

2. Consider relation R from Part 2. Is the decomposition of R into R1(D,E,G) and R2(A,B,C,E,F) in BCNF? Is it a lossless decomposition?

3. Consider a relation R(A,B,C,D) and the functional dependencies that hold over R:
   ABC $\rightarrow$ D
   D$\rightarrow$A

Is the decomposition of R into R1(A,B,C) and R2(B,C,D) in BCNF? Is it a lossless decomposition?

Part 5: Schema refinement in an application

You are designing an application that maintains and analyzes information on product orders. The data is currently stored in the following relation:

```
Orders(OrderID, CustomerID, CustomerName, ProductID,
ProductName, ItemPrice, Quantity)
```

Here is an example instance of this relation:

OrderID	CustomerID	CustomerName	ProductID	ProductName	ItemPrice	Quantity
1	100	Alice	10	Widget	9.99	2
2	100	Alice	20	Gadget	14.99	1
3	101	Bob	10	Widget	9.99	5
4	101	Bob	30	Doohickey	4.99	3
5	102	Carol	20	Gadget	14.99	2

Think about all the non-trivial FDs that hold in this application and schema, and write them down.

Q3: Using the FDs you identified, decompose Orders into a new set of tables that are in BCNF. Use a SQL statement like the following (see also example in Part 4):

```
CREATE TABLE <YourTableName> AS ( ... )
```

Submit all your CREATE TABLE statements in the file Q3.sql.

The file should only contain CREATE TABLE statements like the one above; do not drop tables or inject other commands.
Hint: make sure that your new tables do not contain redundant information by repeating the same tuple several times.

Discussion: Can you see any reason why you may prefer to not force your schema to conform to BCNF in this application? Consider what useful queries may become slower due to the decomposition.